

Memoria del proyecto StrongerThings

Bitácora de la evolución, decisiones técnicas y aprendizajes durante el desarrollo del proyecto.

Esta memoria documenta el camino recorrido desde el proyecto base **mongo-Rol** hasta llegar a una aplicación full-stack con autenticación, almacenamiento en la nube, vista de administración, sistema de hechizos completo de D&D 5e, hoja de personaje imprimible, modo oscuro, responsive móvil, reorganización en monorepo `client/ + server/`, editor de mapas tácticos con generación por IA y sistema de control de acceso por features. Cada hito recoge la motivación, las decisiones que se tomaron y los problemas reales que aparecieron en el camino.

Índice

1. [Punto de partida](#)
 2. [Adaptación a D&D 5e](#)
 3. [Dockerización y problemas de red](#)
 4. [Autenticación con JWT y bcrypt](#)
 5. [Vista de administrador y roles](#)
 6. [Subida de hojas de personaje](#)
 7. [Front con React + Vite](#)
 8. [Refactor: edición y operaciones avanzadas](#)
 9. [Limpieza, README y documentación](#)
 10. [Sistema de hechizos completo \(D&D 5e\)](#)
 11. [Hoja de personaje imprimible](#)
 12. [Modo oscuro y toggle de tema](#)
 13. [Responsive y experiencia móvil](#)
 14. [Toasts, ErrorBoundary y robustez](#)
 15. [Diario por personaje y página global](#)
 16. [Catálogos: buscadores y filtros](#)
 17. [Refactor de estructura: monorepo y atomización CSS](#)
 18. [Lecciones aprendidas \(v1\)](#)
 19. [Google OAuth 2.0](#)
 20. [Avatares de personaje](#)
 21. [Campañas y sesiones](#)
 22. [Bestiario: monstruos del SRD](#)
 23. [Editor de mapas tácticos con react-konva](#)
 24. [Tiles PNG y assets Kenney](#)
 25. [Tokens con imagen circular](#)
 26. [Generación de mapas con IA \(Claude Sonnet\)](#)
 27. [aiMeta: guardar el prompt para regenerar](#)
 28. [Sistema de feature flags por usuario](#)
 29. [Panel de administración ampliado](#)
 30. [Lecciones adicionales \(v2\)](#)
-

1. Punto de partida

Base: proyecto académico `mongo-Rol`, una API simple de gestión de personajes con MongoDB y Mongoose.

Estado inicial:

- Express con un único modelo `Character` y otro `Object`.
- Sin autenticación. Sin separación entre lógica de negocio y controladores.
- Sin Docker, sin tests, sin documentación.

Decisión: usar mongo-Rol como esqueleto para construir un proyecto más ambicioso temáticamente: un gestor de personajes de **Dungeons & Dragons 5e** con todas las features que normalmente faltan en un proyecto académico (auth, roles, almacenamiento externo, front, deploy).

2. Adaptación a D&D 5e

Decisiones de modelado

Los modelos genéricos del proyecto base no representaban bien las reglas reales del juego. Se rediseñaron para reflejar D&D 5e con fidelidad:

- `User` : nuevo. `username`, `email`, `password_hash` (luego `role`).
- `BaseObject` : catálogo compartido de objetos del juego. Stats típicos de D&D (`damage`: "1d8", `armorClass`, `properties`: ["versatile"], `rarity`, `requiresAttunement`).
- `Character` : ahora con `charClass` (enum de 13 clases), `race` (enum), `level` (max 20), `gold`, `abilityScores` (los 6 atributos clásicos: STR/DEX/CON/INT/WIS/CHA), `hitPoints` y un `inventory` como array de **documentos embebidos**.

El patrón de inventario

Decisión clave: el inventario no almacena objetos completos, sino **instancias** que referencian al catálogo. Cada item tiene `baseObject` (`ObjectId`), `customName`, `quantity`, `durability`, `equipped`, `notes`.
Ventajas:

- Un mismo objeto del catálogo (ej. "Espada Larga") puede ser instanciado N veces con stats personalizados.
- Si se actualiza el catálogo, los items existentes heredan los cambios.
- `populate()` resuelve la referencia cuando el cliente lo pide.

Estructura de carpetas adoptada

Siguiendo principios de separación de responsabilidades:

```
src/
├─ config/           Conexión a infraestructura
├─ models/           Esquemas Mongoose
├─ services/         Lógica de negocio pura (sin Express)
├─ controllers/      Manejo HTTP/JSON
├─ routes/           Definición de endpoints
└─ middlewares/      Validaciones, auth, errores
```

Esta separación demostró ser muy útil más adelante: cuando se cambió de almacenamiento local a Cloudinary, sólo hubo que tocar el `service` correspondiente, no los controladores.

3. Dockerización y problemas de red

Se añadió Docker Compose con dos servicios: `app` y `mongo`. **Aquí aparecieron las primeras dificultades reales del proyecto.**

Problema 1: hostnames distintos según el entorno

Dentro de Docker, los contenedores se hablan por nombre de servicio (`mongo` o `monger_things`). Pero cuando se ejecuta `npm run dev` desde la máquina host, ese nombre no se resuelve — hay que usar `localhost` .

Error típico: `EAI_AGAIN monger_things` al hacer `npm run dev` .

Solución adoptada: sobrescribir las variables en el bloque `environment:` del `docker-compose.yml` . Así el `.env` queda configurado para desarrollo local, y Docker pisa lo necesario al levantar el stack:

```
environment:
  MONGO_HOST: monger_things    # nombre del contenedor (solo dentro de Docker)
  MONGO_PORT: 27017           # puerto interno de Mongo
```

Problema 2: puertos internos vs expuestos

Docker mapea `27027:27017` — el primero es el del host, el segundo el interno.

Dónde corre	MONGO_HOST	MONGO_PORT
Dentro de Docker	monger_things	27017
npm run dev en host	localhost	27027

Problema 3: el orden de carga de `dotenv`

Este fue de los más sutiles y costosos del proyecto. El error: aunque el `.env` tenía las variables correctas, dentro del código aparecían como `undefined` .

Causa: en ES Modules, los `import` se evalúan **antes** que el código del archivo. Si `dotenv.config()` está en línea 5 pero `import "./config/upload.js"` está en línea 4, cuando `upload.js` lee `process.env.X` aún no ha corrido `dotenv`.

Solución definitiva: mover la carga de `dotenv` a su propio módulo y hacer que sea el **primer import** del entry point:

```
// src/config/loadEnv.js
import dotenv from "dotenv";
dotenv.config();

// src/index.js
import "./config/loadEnv.js"; // PRIMERA línea – carga el .env antes que nada
import express from "express";
// resto de imports...
```

Este patrón es el que se ve en proyectos serios con ESM. **Aprendizaje importante** que vale para cualquier proyecto Node con módulos ES.

4. Autenticación con JWT y bcrypt

Se añadió un sistema de autenticación completo:

- `/api/auth/register` — crea usuario, hashlea password con bcrypt (10 rounds), devuelve token.
- `/api/auth/login` — verifica credenciales y devuelve `{ user, token }`.
- `/api/auth/me` — endpoint protegido para obtener el usuario actual.

Middleware `authRequired`

Lee la cabecera `Authorization: Bearer <token>`, verifica el JWT, busca el usuario en la base y lo deja en `req.user` para los handlers siguientes. Errores 401 con mensajes diferenciados (token ausente, inválido, expirado).

Scoping por usuario

Una vez había auth, cada personaje pasó a estar **vinculado a su creador**. Las queries se filtran por `req.user._id`:

```
Character.find({ user: userId })
```

Con `403 Forbidden` si alguien intenta acceder a un personaje ajeno (`character.user.toString() !== userId.toString()`).

Lección de seguridad: nunca exponer el hash

Se sobrescribió `toJSON` en el modelo `User`:

```
userSchema.methods.toJSON = function () {
  const obj = this.toObject();
  delete obj.password_hash;
  return obj;
};
```

Así, aunque cualquier endpoint devuelva un user, el cliente nunca recibe el hash.

5. Vista de administrador y roles

Se añadió un sistema de roles (`user / admin`) y una vista exclusiva para admins.

Backend

- Campo `role` en el modelo `User`, con default `"user"`.
- Middleware `adminRequired` que comprueba `req.user.role === "admin"`.
- Endpoints en `/api/admin/*`: estadísticas, listar usuarios, cambiar rol, eliminar usuario.
- **Borrado en cascada**: al eliminar un user se borran todos sus personajes (`Character.deleteMany({ user: id })`).

Frontend

- Componente `<AdminRoute>` que redirige a `/characters` si el usuario no es admin.
- Página `/admin` con tabla de usuarios, botones de promover/degradar/eliminar y panel de estadísticas.
- Enlace "🔑 Admin" en el header solo visible para admins.

El primer admin

No se puede crear el primer admin desde la API (no hay nadie con permisos para hacerlo). Solución: promoverse manualmente desde MongoDB Compass o `mongosh` la primera vez. A partir de ahí, los siguientes se pueden gestionar desde la UI.

6. Subida de hojas de personaje

La feature más turbulenta del proyecto. Pasamos por **tres iteraciones** antes de quedarnos con la solución final.

Iteración 1: almacenamiento local

`multer.diskStorage` guardando los PDFs en `uploads/character-sheets/` con UUID como nombre. Funcional pero limitado: los archivos no sobreviven a un `docker compose down -v` y no son accesibles desde fuera.

Iteración 2: Google Drive con Service Account

Tema bonito sobre el papel: usar la API de Google Drive con una Service Account y subir los archivos a una carpeta compartida.

Setup: crear proyecto en Google Cloud, activar Drive API, crear Service Account, descargar JSON de credenciales, compartir la carpeta de Drive con el email de la SA.

El error definitivo:

```
Service Accounts do not have storage quota.
Leverage shared drives, or use OAuth delegation instead.
```

Google cambió su política y las Service Accounts ya no pueden crear archivos en Drive personal, sólo en Shared Drives (que requieren Workspace de pago) o usando OAuth delegation (más complejo).

Aprendizaje involuntario: aunque tengas permiso de Editor en una carpeta, sin **cuota propia de almacenamiento** Google rechaza la creación.

Iteración 3: Cloudinary (la definitiva)

Cambio de proveedor a Cloudinary. Plan gratuito generoso (25 GB), API simple, soporte nativo de PDFs.

Setup minimalista: cuenta gratuita (sin tarjeta), tres credenciales (`cloud_name` , `api_key` , `api_secret`), activar entrega de PDFs en Settings → Security.

Cambios en código mínimos gracias a la separación en services. Multer pasó a `memoryStorage()` (recibe buffer), `cloudinaryService.js` lo sube vía `upload_stream` , y se almacena en BD el `cloudinaryUrl` para descargas futuras.

Para que el PDF se sirviera **inline en lugar de descargar**, hubo que subirlo como `resource_type: "image"` con `format: "pdf"` (Cloudinary trata así los PDFs como imagen vectorial). Permite previsualizarlo en un `<iframe>` .

Lecciones de esta saga

1. **Las APIs gratuitas tienen letra pequeña.** Lee siempre la política antes de invertir setup.
2. **La arquitectura limpia paga.** Cambiar de Drive a Cloudinary fue cuestión de reescribir un único service. Los controladores y rutas no se tocaron.

3. **Saber cuándo abandonar.** Tras dos horas peleando con Service Accounts, el cambio a Cloudinary fue cuestión de 30 minutos.

Coletilla: incidente de seguridad

Durante la fase de Drive, el archivo `google-credentials.json` se commiteó accidentalmente. GitHub lo detectó y bloqueó el push con su sistema de secret scanning.

Resolución: se revocó la Service Account inmediatamente en Google Cloud, se reescribió el historial con `git filter-repo --invert-paths --path google-credentials.json`, y se hizo force push.

Lección: GitHub tiene secret scanning activo, lo cual es genial.

7. Front con React + Vite

Tras tener un backend completo, llegó el front.

Decisiones técnicas

- **React + Vite** sobre HTML/JS vanilla por: hot reload, componentes, routing, state management con hooks.
- **Sin framework UI** (Material-UI, Chakra, etc.): CSS propio con tema D&D personalizado. Quería una estética con identidad ("pergamino") en vez de una app más con cards genéricas.
- **Fetch nativo**, no Axios: una dependencia menos. Wrappers simples en `api/client.js`.
- **Context API** para auth (no Redux ni Zustand): el estado es pequeño y no compensa la complejidad.

Estética: dos temas

- **Pergamino** (light): paleta crema/sepia/oro, fuente Cinzel + EB Garamond + MedievalSharp.
- **Dungeon** (dark): paleta negro/dorado tenue, mismas tipografías.

Toggle en el header. Persistencia con `localStorage`. Variables CSS por `data-theme="..."` en el `<html>`.

CORS

Primer obstáculo del front: el navegador bloqueaba las peticiones a `localhost:3001` desde `localhost:5173`. Solución: instalar `cors` en el backend con la URL del front en el origen permitido:

```
app.use(cors({
  origin: process.env.FRONTEND_URL,
  credentials: true
}));
```

8. Refactor: edición y operaciones avanzadas

Una vez lo básico funcionaba, se añadieron features de calidad de vida.

Edición inline

Click sobre cualquier número del personaje (PG, oro, nivel, atributos) → input editable → Enter guarda, Esc cancela. Componente `<EditableNumber>` reutilizable, basado en estado local + onSave callback.

Detalle UX: los valores tienen un subrayado punteado para indicar que son editables. Es la convención de "click para editar" sin gritarle al usuario.

Operaciones del inventario

Al principio sólo se podían añadir/quitar items y equipar/desequipar. Se añadieron:

- **Slider de durabilidad** en tiempo real.
- **Modal de edición completo** con todos los campos del item (cantidad, customName, durabilidad, equipped, notes).
- **Sub-rutas en el backend:** `PUT /api/characters/:id/inventory/:itemId` y `DELETE` equivalente.

Visor de PDF

Modal con `<iframe>` que carga la URL de Cloudinary directamente. La key fue subir el PDF con `resource_type: "image"` para que Cloudinary lo sirviera inline.

Aumento del límite de tamaño

Pasó de 5 MB a 50 MB. Se ajustaron tres sitios: `Multer ( limits.fileSize )`, `Express ( express.json( { limit: ... } )` y el mensaje de error en el handler.

Unificación de "Editar" y "Ver hoja"

Al principio había dos vistas distintas del personaje: un mini-form de edición dentro de `CharactersPage` (con 5 campos básicos), y una página de detalle `CharacterDetailPage` con pestañas, mucho más completa. Era redundante.

Decisión: unificar. El botón "Editar" pasó a ser un `Link` a `/characters/:id?edit=1`. La pestaña General lee el query param y arranca directamente en modo edición. Tras guardar o cancelar, se limpia el param con `setSearchParams(..., { replace: true })` para que un refresh no reabra el form.

Patrón limpio: **el componente decide su estado inicial leyendo la URL**, sin tener que pasar props desde arriba.

9. Limpieza, README y documentación

Última fase de la primera iteración: dejar el proyecto presentable.

- README completo con setup paso a paso, incluyendo la creación de cuenta en Cloudinary.
- `.env.example` con valores ficticios para que cualquiera pueda clonar y arrancar.
- Esta MEMORIA.
- Limpieza de archivos sin uso (`script test.js` heredado del base, restos de Drive).
- `.gitignore` repasado: `node_modules/`, `.env`, `uploads/` (por si acaso), `google-credentials.json`.

10. Sistema de hechizos completo (D&D 5e)

Tras la versión 1, faltaba lo más característico del juego: la magia. Esta fase fue el bloque más grande de trabajo posterior y tocó modelo, servicio, controlador, rutas, semilla, varias páginas del front y la hoja imprimible.

Modelado: Spell + spellcasting embebido

Dos piezas separadas:

`Spell` (modelo independiente): catálogo maestro compartido entre todos los usuarios. Un solo documento por hechizo. Campos: `name`, `nameOriginal` (inglés), `level` (0-9), `school` (Evocación, Necromancia...), `castingTime`, `range`, `duration`, `concentration`, `ritual`, `components`: {verbal, somatic, material, materialDesc}, `damageType`, `classes` (qué clases pueden lanzarlo), `description`, `atHigherLevels`.

`spellcasting` dentro del `Character` (subschema): el estado mágico **del personaje**, no del hechizo.

Campos:

- `ability`: el atributo lanzador (INT/WIS/CHA).
- `attackBonus`, `saveDC`: estadísticas mágicas.
- `spellSlots`: matriz nivel1..nivel9 con {total, used}. Los espacios consumibles para lanzar.
- `spellsKnown`: array de {spell, prepared, notes}. Cada entrada referencia un `Spell` del catálogo.

Es una separación importante: el hechizo es público, los espacios consumidos son privados.

Semilla con la API oficial

Se escribió un script `seed-spells.js` que descarga el catálogo de hechizos de la **D&D 5e API oficial** (dnd5eapi.co), traduce los campos al español y los inserta. La primera vez popula la BD con ~300 hechizos sin tener que escribirlos a mano.

Endpoints

```
GET    /api/spells           (con ?search, ?level, ?class)
GET    /api/spells/:id
POST   /api/spells
DELETE /api/spells/:id

POST   /api/characters/:id/spells      Aprender un hechizo
PATCH /api/characters/:id/spells/:knownId  Editar (prepared, notes)
DELETE /api/characters/:id/spells/:knownId  Olvidar
```

Frontend

- **Pestaña Conjuros** en el personaje (`SpellsSection`): aptitud mágica, slots, listado de aprendidos por nivel con detalle expandible, modal de selección desde el catálogo.
- **Página /spells** (`SpellsPage`): catálogo navegable global. Filtros por nivel/clase/escuela, búsqueda por nombre con mínimo de 3 caracteres y debounce, ordenación por nivel (agrupado con cabeceras) o alfabético (lista plana con badge de nivel).
- Botón "+ Nuevo hechizo" con formulario completo (todos los campos del modelo, V/S/M, clases compatibles via toggles).
- Eliminar **solo visible para admins** desde el cliente, porque el catálogo es compartido.

Lección de UX: el catálogo es compartido

El catálogo `/api/spells` no es por usuario. Si un usuario borra un hechizo se va para todos los demás. El backend lo permite a cualquier autenticado, pero el frontend solo muestra el botón a admins. Cuando se decida tener miles de usuarios, esto se reforzará añadiendo `adminRequired` al endpoint `DELETE`.

11. Hoja de personaje imprimible

Una de las features más visuales. El usuario podía ya editar todo el personaje en pantalla, pero faltaba una forma de **llevarse a la mesa de juego en papel** o exportarlo a PDF.

Diseño multi-página A4

Componente `CharacterSheetPrint` (ruta `/characters/:id/print`). Tres páginas A4 fijas:

- **Página 1:** stats (los 6 atributos), competencias, salvaciones, habilidades, combate (CA/iniciativa/velocidad/HP), dados de golpe, salvaciones contra muerte, oro, tabla de ataques, equipo.
- **Página 2:** personalidad (rasgos, ideales, vínculos, defectos, historia, aspecto, aliados, tesoro) e información física (edad, altura, peso, ojos, piel, pelo). Idiomas y otras competencias al final.
- **Página 3** (solo si tiene magia): aptitud mágica, espacios de conjuro en grid 3×3, lista de hechizos conocidos por nivel.

Las medidas son en `mm` y `pt` para que el resultado impreso sea fiel: `width: 210mm`, `min-height: 297mm`, fuentes en puntos (8pt, 10pt, 18pt). El navegador respeta los `mm` literalmente al imprimir o exportar a PDF.

Toggles en la barra superior

Dos checkboxes encima del botón "Imprimir":

1. **"Vacía para rellenar a mano":** vacía los campos numéricos/mecánicos y los sustituye por una raya `_____`. La identidad, narrativa, físico, inventario y lista de hechizos se mantienen rellenos. Útil para imprimir y dejar a un jugador novato.
2. **"Incluir avatar al imprimir":** por defecto desactivado para mantener la hoja "clásica". Si se activa, aparece el avatar circular en la cabecera de la página 1.

El estado vive en `useState` dentro del componente. El `@media print` oculta los toggles para que no salgan en el papel.

El problema del `@media print` y el navbar

Al pulsar "Imprimir/Guardar como PDF", la cabecera de la app (el `<Header>` con el logo, los enlaces "Personajes/Catálogo/Hechizos") aparecía en la primera página, rompiendo el diseño A4.

Causa: el `@media print` original sólo ocultaba `.no-print` y `.sheet-controls`, pero el header de la app no llevaba ninguna de las dos.

Solución: añadir `.app-header` a la lista de `display: none !important` dentro del `@media print`. Una línea, problema resuelto.

Forzar tema claro al imprimir

El usuario podía estar en modo oscuro al imprimir. Si las variables `--parchment` y `--ink` apuntaban a colores oscuros, la hoja salía negra sobre negra. Solución: dentro del `@media print` se fuerzan los tokens originales:

```
@media print {
  :root[data-theme="dark"] {
    --parchment: #f4e8d0;
  }
}
```

```
    --ink: #2b1810;
  }
}
```

12. Modo oscuro y toggle de tema

El tema "dungeon" oscuro se implementa por sobreescritura de variables CSS con el selector `:root[data-theme="dark"]`. El `<ThemeContext>` aplica el atributo al `<html>` y persiste en `localStorage`.

El bug del navbar invisible

Inicialmente, al activar el tema oscuro la cabecera quedaba con texto oscuro sobre fondo oscuro: prácticamente invisible salvo al pasar el cursor por encima (que usa `--gold-bright`).

Causa: el header usaba `color: var(--parchment)` para los enlaces del nav. En tema claro `--parchment` es crema (`#f4e8d0`), pero en tema oscuro la variable se redefine a `#2a2520`. Resultado: texto oscuro sobre el fondo cuero del header, que en oscuro también baja a `#1a1410`.

Solución: añadir overrides específicos `:root[data-theme="dark"] .app-header *` con colores **fijos** (no variables). En particular el texto en `#e8d8b8` y el botón "Salir" como outline claro.

Lección: las variables semánticas (`--parchment`, `--ink`) son útiles, pero hay zonas (header oscuro siempre) donde tiene más sentido un color fijo o una variable dedicada al rol (`--header-text`).

Transiciones suaves entre temas

Pequeño detalle visual: al pulsar el toggle, los colores de `body`, `.scroll-card`, `.btn`, inputs, etc. se transicionan con `transition: background-color 0.25s, color 0.25s, border-color 0.25s`. Evita el "flash" de cambio brusco.

La hoja imprimible (`.sheet-page`) tiene `transition: none` explícitamente para que no se anime durante una impresión iniciada justo después de cambiar tema.

13. Responsive y experiencia móvil

La app empezó pensada para desktop. Adaptarla a móvil/tablet fue una fase específica con varios subproblemas.

Breakpoints decididos

- $\leq 1024\text{px}$ → tablet apaisado, ajustes suaves.
- $\leq 900\text{px}$ → tablet vertical / móvil apaisado. **Aquí aparece el menú hamburguesa.**
- $\leq 768\text{px}$ → móvil grande.
- $\leq 480\text{px}$ → móvil pequeño.

Menú hamburguesa

En `Header.jsx` se añadió un botón con tres `<span>` que se transforman en X cuando se abre. Estado en `useState`, cierre automático al cambiar de ruta (`useLocation` + `useEffect`) y al pulsar `Escape`.

El nav en móvil se posiciona en `absolute` debajo del header, con `max-height: 0` colapsado y `max-height: 80vh` desplegado, con transición de altura suave. Accesibilidad básica con `aria-expanded`,

`aria-controls` y `aria-label`.

Las pestañas del editor con sticky

Una mejora muy útil: la barra de pestañas del personaje (`.tabs-nav`) pasó a ser `position: sticky` para que al hacer scroll dentro de "Atributos" o "Personalidad" siguiera visible y se pudiera cambiar de sección sin volver al inicio.

Bug: el sticky se rompía al cambiar de pestaña

Síntoma: el sticky funcionaba al cargar la página, pero al cambiar de pestaña dejaba de pegarse "a veces". Era impredecible.

Causa: el contenedor de la pestaña activa tenía `animation: fadeIn` con `transform: translateY(...)` en los keyframes. Cualquier ancestro con un `transform` aplicado crea un nuevo **containing block** que cancela el `position: sticky` de los descendientes.

Fix: quitar el `transform` de la animación, dejándola solo con `opacity` :

```
@keyframes fadeIn {
  from { opacity: 0; }
  to   { opacity: 1; }
}
```

La animación sigue funcionando (fade suave de 0.2s) y el sticky deja de romperse.

Lección clave: `position: sticky` es muy sensible. Cualquier ancestro con `transform`, `filter`, `will-change` u `overflow: hidden` lo rompe silenciosamente.

Las pestañas en móvil: scroll horizontal en vez de wrap

En móvil con 7 pestañas, `flex-wrap: wrap` las repartía en 2-3 filas y la barra se comía demasiado vertical.

Solución: en `@media (max-width: 768px)`, `flex-wrap: nowrap` + `overflow-x: auto`. La barra queda en una sola fila y el usuario desliza horizontalmente.

Pequeño detalle visual: un `mask-image` con gradiente a transparente en el borde derecho sugiere visualmente que hay contenido al que llegar deslizando.

14. Toasts, ErrorBoundary y robustez

A medida que el proyecto crecía, hacía falta un sistema más uniforme de feedback de acciones.

Sistema de toasts

Componente `<Toast>` + contexto `ToastContext` con API:

```
const toast = useToast();
toast.success("Personaje creado");
toast.error("Error al borrar", "Inténtalo de nuevo");
toast.warning("Hay cambios sin guardar");
toast.info("Sesión iniciada");
```

Cuatro variantes con icono y color: success (verde), error (rojo), warning (dorado), info (azul). Auto-cierre a los 3s (6s para errores), o `duration: 0` para no cerrarse solo. Animación slide-in desde la derecha.

Los toasts viven en un `<div class="toast-container">` con `position: fixed` en la esquina superior derecha, `z-index: 9999` para estar por encima de modales.

ErrorBoundary global

Componente de clase (los hooks de React aún no soportan `componentDidCatch`) que envuelve el `<App>` en `main.jsx`. Si algún componente revienta durante el render, en lugar de mostrar una pantalla en blanco, aparece una página de error con:

- Icono y mensaje genérico.
- Mensaje del error.
- Stack trace en `<details>` (solo en desarrollo, `import.meta.env.DEV`).
- Botones "Volver al inicio" y "Recargar página".

Es una red de seguridad: en una app pequeña los crashes son raros, pero cuando ocurren el usuario debe ver algo informativo, no un blanco.

15. Diario por personaje y página global

Feature narrativa: cada personaje puede llevar un diario de aventuras.

Modelo

Subschema `diaryEntrySchema` con `title`, `date`, `content`, embebido como array `diary` en el `Character`. `_id: true` para poder editar/borrar entradas individuales. `timestamps: true` automático.

```
diary: { type: [diaryEntrySchema], default: [] }
```

Endpoints

```
POST    /api/characters/:id/diary
PUT     /api/characters/:id/diary/:entryId
DELETE  /api/characters/:id/diary/:entryId
```

Validación en backend (contenido no vacío), validación de ownership (`character.user === req.user._id`), `notFound("Entrada de diario no encontrada")` si el `entryId` no existe.

Pestaña Diario en el personaje

Componente `<DiarySection>`. Lista de tarjetas colapsables ordenadas por fecha (más recientes arriba). Click para expandir el contenido. Modo edición inline con form. Borrado con confirmación.

Página global /diary

Vista agregada de **todas las entradas de todos los personajes del usuario** en modo solo lectura. Útil cuando el usuario tiene varios personajes y quiere ver su actividad reciente sin entrar en cada uno.

Implementación: reutiliza `GET /api/characters` (que ya devuelve los personajes con su `diary`), aplana en cliente con `useMemo` y filtra por personaje (dropdown) y texto (búsqueda en título y contenido). Cada

tarjeta muestra el avatar mini del personaje y un enlace para ir a editar la entrada en el personaje original.

Decisión técnica: no se creó un endpoint dedicado `/api/diary` porque el aplanado en cliente es trivial. Si el catálogo crece a docenas de personajes con cientos de entradas cada uno, se moverá al backend (paginación, filtros server-side).

Naming: la página global se llama "Crónicas" en el menú y la pestaña por personaje "Diario". Distinción semántica: una persona tiene **un** diario; varios diarios juntos son **crónicas**.

16. Catálogos: buscadores y filtros

Tanto el catálogo de objetos como el de hechizos crecieron a varios cientos de entradas. Hacía falta poder navegarlos.

Buscador del catálogo de objetos

Barra `<form>` envuelta en `.scroll-card` por encima de los botones de categoría. Filtrado en cliente (el catálogo ya está cargado), búsqueda case-insensitive por nombre, botón `x` para limpiar visible solo cuando hay texto, contador "Mostrando X de Y objetos" debajo.

El filtro de categoría y la búsqueda funcionan como AND.

Filtros del catálogo de hechizos

Cuatro filtros + ordenación, todos sincronizados:

- **Búsqueda por nombre:** enviada al backend como query param `?search=` con **mínimo 3 caracteres** (evita peticiones inútiles con 1-2 letras) y **debounce de 250ms** (espera a que el usuario deje de teclear).
- **Filtro de nivel:** enviado al backend.
- **Filtro de clase:** enviado al backend.
- **Filtro de escuela:** aplicado en cliente.
- **Ordenación:** por nivel (agrupado con cabeceras "Nivel 0", "Nivel 1"...) o alfabético (lista plana con badge de nivel en cada tarjeta).

Detalle de UX: el campo "Buscar" usa la clase `.field--search` que en móvil ocupa `span 2` (ancho doble) para destacar visualmente que es el principal.

17. Refactor de estructura: monorepo y atomización CSS

A medida que el proyecto crecía, dos cosas pedían reorganización: la estructura del repo y el CSS gigante.

Monorepo `client/ + server/`

Antes: el backend en un repo y el frontend en otro hermano, sin nada que los relacionara.

Después: un único repo `Stronger-Things` con dos subcarpetas:

```
Stronger-Things/
├─ client/      ← antes Stronger-Things-Front
├─ server/      ← antes en la raíz
├─ README.md
└─ .gitignore
```

Migración: `git mv src server/src`, `git mv package.json server/package.json` y resto de archivos del backend a `server/`. Como `git mv` registra los movimientos como **rename**, el historial de cada archivo se conserva intacto. Un solo commit con todos los renames.

`node_modules/` y `.env` se reinstalan/mueven a mano porque no están en git. Tras el move se hace `cd server && npm install` y `cd ../client && npm install`.

Atomización del CSS

El `global.css` había llegado a ~2200 líneas en un solo archivo. Encontrar cualquier regla requería Ctrl+F, y los conflictos eran difíciles de detectar.

Decisión: dividir en **17 archivos** organizados por capas, importados desde un único `index.css` :

```
client/src/styles/
├─ index.css           ← entry point, solo @imports
├─ tokens.css          ← variables :root
├─ base.css            ← reset, body, tipografía
├─ layout.css          ← container, grid, header, scroll-card
├─ forms.css           ← form, btn, alert, empty, loading
├─ components/
│   ├─ stats.css
│   ├─ inventory.css
│   ├─ narrative.css
│   ├─ chips.css
│   ├─ spells.css
│   ├─ tabs.css
│   ├─ modals.css
│   ├─ toasts.css
│   ├─ error-boundary.css
│   ├─ override-popover.css
│   └─ theme-toggle.css
├─ pages/
│   ├─ auth.css
│   └─ sheet-print.css ← toda la maquetación A4
├─ themes/
│   └─ dark.css        ← todos los overrides de tema oscuro
├─ responsive.css      ← media queries por breakpoint
└─ print.css           ← @media print
```

Verificación rigurosa: se contaron los selectores antes y después. **207 selectores en el original, 207 en la estructura nueva** — cero pérdidas.

Reorganización de `components/`

`components/` también estaba aplanado. Quedó:

```
client/src/components/
├─ ui/                ← componentes "tontos" reutilizables
│   ├─ ErrorBoundary.jsx
│   ├─ Toast.jsx
│   └─ PDFPreviewModal.jsx
├─ layout/            ← navegación, rutas, estructura
```

```
|   |— Header.jsx
|   |— ThemeToggle.jsx
|   |— Tabs.jsx
|   |— ProtectedRoute.jsx
|   |— AdminRoute.jsx
|   |— character/ ← ya existía, se mantiene
|   |— *Section.jsx
```

Cuatro archivos actualizan sus imports: `main.jsx` , `App.jsx` , `ToastContext.jsx` , `CharacterDetailPage.jsx` . Cinco minutos de cambios.

18. Lecciones aprendidas (v1)

Sobre la arquitectura

- **La separación service/controller/routes paga muy rápido.** Cualquier cambio de infraestructura (ej. Drive → Cloudinary) toca un sólo archivo.
- **Los middlewares son tu amigo.** `authRequired` , `adminRequired` , `validateBody` , `validateObjectId` aplicados con `router.use(...)` ahorran muchísimo código.
- **Usa enum siempre que sea posible** en Mongoose. Pillar errores de tipeo antes de que lleguen a base.
- **Subschemas embebidos vs colecciones separadas:** para inventario y diario tiene sentido embeber (siempre se cargan junto al personaje). Para hechizos no (es un catálogo compartido). La pregunta es: *¿este dato existe por sí mismo o solo en el contexto del padre?*

Sobre Node.js y ES Modules

- **El orden de imports importa.** Si un import lee `process.env` , asegúrate de cargar `dotenv` ANTES, idealmente en su propio módulo.
- `process.env.X` **siempre llega como string.** `=== "true"` , no `=== true` . Las flags booleanas en `.env` son strings.
- `--env-file=.env` de Node 20+ es una alternativa a `dotenv` . Útil para scripts puntuales.

Sobre Docker

- **Las variables del `.env` no se pasan automáticamente al contenedor.** Hay que ponerlas en el bloque `environment:` del `compose` .
- `depends_on` **no espera a que el servicio esté listo**, sólo a que el contenedor exista. Para esperas reales, usar `healthchecks` o un `retry` en el cliente.
- **Los volúmenes nombrados** (`mongo_data`) sobreviven a `docker compose down` , no a `down -v` . Cuidado al limpiar.

Sobre seguridad

- **Nunca commitar secretos.** Aunque el repo sea privado.
- **Si pasa, revocar primero, reescribir historial después.** El historial reescrito no devuelve la clave a su estado seguro — sólo la quita del repo.
- **GitHub Secret Scanning funciona.** Mejor confiar en él como red de seguridad.
- **Las Service Accounts de Google no son la solución para todo.** Para Drive personal hay limitaciones de cuota que no son obvias.

Sobre CSS y diseño

- **Las variables semánticas tienen límites.** Cuando un componente debe verse igual en ambos temas (header oscuro, hoja imprimible clara), forzar colores fijos es más limpio que pelearse con la cascada de variables.
- **position: sticky es frágil.** Cualquier ancestro con `transform`, `filter`, `will-change` u `overflow: hidden|auto` lo rompe sin avisar.
- **Las animaciones con transform en keyframes afectan al posicionamiento de descendientes.** Si quieres `fadeIn` cerca de elementos sticky, hazlo solo con `opacity`.
- **@media print y @media (max-width: ...) son contextos distintos.** El navegador no aplica responsive al imprimir; el A4 manda.
- **Atomizar el CSS reduce errores.** Pasar de 2200 líneas en un archivo a 17 archivos de 50-500 líneas hace que localizar cualquier regla sea trivial.

Sobre el frontend

- `useSearchParams` **para estado-en-URL.** Cuando un estado debe poder linkarse (modo edición, filtros), guardarlo en query params hace que sea compartible y resistente a refresh.
- **Context API basta para casi todo.** Auth, tema y toasts caben perfectamente sin Redux ni Zustand.
- `useMemo` **para aplanados costosos.** En la página `/diary` el aplanado de todas las entradas de todos los personajes se memoriza con `useMemo`, no se rehace en cada render.
- **Optimistic UI cuesta poco y se nota mucho.** Para futuras versiones, aplicar los cambios al state local antes de esperar al backend daría más sensación de instantaneidad.

Sobre el desarrollo en sí

- **El copy-paste desde un chat puede romper código.** Los enlaces Markdown se quedan pegados (`[process.env.X](http://process.env.X)` en lugar de `process.env.X`). Mejor descargar archivos planos o teclear a mano las líneas críticas.
- **Iterar pequeño, validar pronto.** Cada feature se probó en Postman/Vite antes de añadir más capas.
- **Saber cuándo cambiar de enfoque.** Insistir con Drive cuando Cloudinary era 10x más simple no habría aportado nada al proyecto.
- **Refactorizar antes de añadir más cosas.** Cuando el CSS llegó a 2200 líneas, parar y atomizarlo costó una tarde y ahora cada feature nueva es más rápida de añadir.
- **Verificar con métricas, no con vista.** Tras atomizar el CSS, contar `grep -c '^\. [a-zA-Z]'` archivo selectores en el original y la suma de los nuevos archivos confirma que no se ha perdido nada. La impresión visual no basta.

19. Google OAuth 2.0

Motivación

El flujo local (email + contraseña) ya funcionaba, pero añadir OAuth con Google permite registrarse en dos clics sin gestionar contraseñas. Es también un ejercicio realista: casi toda app moderna lo tiene.

Decisiones de diseño

No usar Passport.js. Passport es la opción canónica en Node, pero añade una capa de abstracción que complica el control del flujo y el debugging. Se optó por implementar directamente con el SDK oficial `google-auth-library`:

```
const client = new OAuth2Client(process.env.GOOGLE_CLIENT_ID);
const ticket = await client.verifyIdToken({
  idToken: credential,
```



```
    audience: process.env.GOOGLE_CLIENT_ID
  });
  const { email, name, sub: googleId } = ticket.getPayload();
```

El frontend usa `@react-oauth/google` que inyecta el botón de Google y devuelve un `credential` (JWT firmado por Google). Ese token se manda al backend en `POST /api/auth/google`.

Fusión de cuentas

Un usuario puede tener cuenta local Y cuenta de Google con el mismo email. El campo `provider` del modelo `User` maneja los tres estados:

```
"local" → solo email+contraseña
"google" → solo Google (sin password_hash)
"both" → ambas vinculadas
```

Flujo en el backend:

1. Verificar el ID token con Google.
2. Buscar usuario por `email`.
3. Si no existe → crear con `provider: "google"`, `googleId`, sin `password_hash`.
4. Si existe con `provider: "local"` → actualizar a `"both"`, añadir `googleId`.
5. Si existe con `provider: "google"` o `"both"` → login directo.
6. Emitir JWT propio (igual que el login local).

Lección: unificar por email tiene riesgos en sistemas públicos. Para este proyecto con usuarios conocidos es aceptable. En producción real requeriría verificación de email antes de fusionar.

Variables de entorno necesarias

```
GOOGLE_CLIENT_ID=xxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=xxx
VITE_GOOGLE_CLIENT_ID=xxx.apps.googleusercontent.com
```

Nota: Google OAuth no acepta `http://localhost` sin configuración. Hay que registrar `http://localhost:5173` explícitamente en la consola de Google Cloud como "Authorized JavaScript origin" y "Authorized redirect URI".

20. Avatares de personaje

Iteración 1: URL externa

Se añadió un campo `avatarUrl: String` al modelo `Character`. El usuario pegaba una URL de imagen. Simple, pero incómodo: el usuario tiene que alojar la imagen en algún sitio, los links externos mueren.

Iteración 2: Upload a Cloudinary

Reutilizar el mismo servicio de Cloudinary que ya teníamos para los PDFs fue trivial gracias a la separación en capas. Se añadió `POST /api/characters/:id/avatar` y `DELETE /api/characters/:id/avatar`.

Modelo actualizado:

```
avatar:          { type: String, default: "" },      // URL Cloudinary
avatarPublicId: { type: String, default: "" }        // para el delete
```

Renderizado

El avatar aparece en tres sitios: cabecera de `CharacterDetailPage`, card en `CharactersPage` y la hoja imprimible (toggle "Incluir avatar"). La imagen de Cloudinary se transforma con `w_200,h_200,c_fill,g_face` para servir siempre un cuadrado centrado en la cara.

21. Campañas y sesiones

Motivación

Un DM no juega partidas sueltas; juega **campañas** con múltiples **sesiones**. Era la feature más obvia que faltaba para que la app sirviera de verdad en una mesa.

Modelo de datos

```
// Campaign
{
  dm: ObjectId → User,
  name, description, setting,
  status: "active" | "completed" | "paused",
  notes
}

// Session
{
  dm:          ObjectId → User,
  campaign:    ObjectId → Campaign,      // opcional
  map:         ObjectId → Map,           // añadido con el editor de mapas
  name, description, date,
  status:      "planning" | "active" | "completed",
  participants: [{ character: ObjectId, characterName: String }],
  notes
}
```

Decisión de participantes: se embebe el array en la sesión (no en Campaign) porque la composición del grupo puede cambiar sesión a sesión. El campo `characterName` desnormalizado permite mostrar el nombre aunque el personaje se borre luego.

Frontend

`CampaignsPage` — lista de campañas del DM con detalle expandible de cada sesión: estado, fecha, número de participantes, botones de editar/eliminar.

`SessionsPage` — lista de sesiones agrupadas opcionalmente por campaña. Formulario con selector de campaña, selector de participantes y campo de fecha.

22. Bestiario: monstruos del SRD

El catálogo del SRD

D&D 5e tiene una API pública (`dnd5eapi.co`) con todos los datos del SRD bajo CC BY 4.0. Se escribió un script de semilla en dos pasos:

- `npm run seed:download` — descarga los JSONs crudos del SRD a `server/src/data/` .
- `npm run seed:monsters` — procesa `monsters.json` e inserta en MongoDB.

Traducción con IA: el seed puede llamar a Claude (`claude-haiku-4-5` por economía) para traducir campos descriptivos al español. Si `ANTHROPIC_API_KEY` no está definida, se insertan en inglés.

Frontend: BestiaryPage

Filtros: búsqueda por nombre, tipo de criatura (Beast, Undead, Dragon...), rango de CR. La tarjeta de detalle muestra todas las estadísticas de combate y las acciones expandibles. Vista de solo lectura (el SRD es compartido).

Vinculación con tokens de mapa

Cuando se crea un token de tipo "monster" en el editor, se puede buscar en el bestiario y vincularlo. El token guarda `monster: ObjectId` . En el `GET /api/maps/:id` , el campo `tokens.monster` se popula con `name` , `challengeRating` y `type` . El DM ve los stats del monstruo sin salir del mapa.

23. Editor de mapas tácticos con react-konva

Decisión: Canvas vs SVG vs tabla HTML

Opción	Pros	Contras
Tabla HTML	Sin dependencias, CSS familiar	Drag & drop difícil, sin capas, no escala
SVG	Vectorial, accesible	Performance pobre con 600+ elementos
Canvas con react-konva	GPU-accelerated, drag nativo, capas	Requiere <code>--legacy-peer-deps</code> con React 18

Decisión: react-konva. El `--legacy-peer-deps` es un coste asumible. La API de Konva tiene primitivas exactas para lo que se necesita: `Rect` , `Line` , `Group` , `Image` , `Circle` , `Text` .

Las cuatro capas

Stage	
└─ Layer – Terrain	(PNG tiles + fallback color)
└─ Layer – Grid	(líneas semitransparentes) <code>listening={false}</code>
└─ Layer – Walls	(segmentos de pared personalizados)
└─ Layer – Tokens	(fichas arrastrables)

`listening={false}` en la capa Grid evita que las líneas intercepten los eventos del ratón, que deben llegar a Terrain para pintar.

Pintado de celdas

Problema UX: el usuario espera poder arrastrar el ratón para pintar varias celdas.

Solución: `onMouseDown` activa modo pintura, `onMouseMove` (si mouse down) calcula celda bajo cursor y pinta si es distinta a la última pintada, `onMouseUp` desactiva.

```
const col = Math.floor(pos.x / CELL);
const row = Math.floor(pos.y / CELL);
```

Zoom y pan

`Stage` acepta `scaleX`, `scaleY` y `x`, `y`. Zoom con rueda del ratón (centrado en la posición del cursor, limitado entre 0.3x y 4x). Pan con clic derecho + arrastrar.

Herramientas del editor

- **Pintar** — asigna tile seleccionado
- **Borrador** — asigna tile vacío
- **Paredes** — añade/quita segmentos en bordes de celda
- **Tokens** — abre formulario de nuevo token al hacer clic en celda vacía

24. Tiles PNG y assets Kenney

El problema del SVG inline

Inicialmente los tiles eran SVG inline en `tileCatalog.js`. Funcionaba pero el catálogo llegó a ~500 líneas de strings SVG y los tiles no tenían aspecto de dungeon de verdad.

Migración a PNG

Se adoptaron los packs de **Kenney.nl** (CC0, dominio público):

Pack	Contenido
Roguelike/RPG Pack	Suelos de piedra, hierba, tierra, agua, mobiliario
Tiny Dungeon	Suelos alternativos, muros, puertas
Roguelike Caves & Dungeons	Cuevas, muros naturales, agua con orillas

También se usan iconos de **game-icons.net** (CC BY 3.0 / CC0) para los tokens de criaturas.

Los PNGs van en `client/public/tiles/{terrain,walls,furniture,tokens}/` y se sirven como assets estáticos de Vite.

El hook de precarga

Konva necesita un `HTMLImageElement` cargado para renderizar. Si se pasa la URL string directamente, la imagen aparece con un fotograma de retraso. Solución: precargar todas las imágenes al montar el editor.

```
function preloadImages(sources) {
  const [images, setImages] = useState({});
  useEffect(() => {
    if (sources.length === 0) return;
    const loaded = {};
```

```

    let pending = sources.length;
    const done = () => { if (--pending === 0) setImages({ ...loaded }); };
    sources.forEach(({ key, src }) => {
        const img = new window.Image();
        img.onload = () => { loaded[key] = img; done(); };
        img.onerror = done;
        img.src = src;
    });
}, []);
return images;
}

```

El `setImages` acumula en `loaded` y dispara un único re-render cuando el último PNG termina de cargar.

Estructura del catálogo

```

export const TILE_CATALOG = [
  {
    id:      "floor-stone",
    name:    "Suelo de piedra",
    category: "terrain",
    color:   "#5a5a5a",
    img:     "/tiles/terrain/floor-stone.png",
    svg:     TILE_FLOOR_STONE
  },
  // ...
];

export const TILE_BY_ID = Object.fromEntries(TILE_CATALOG.map(t => [t.id, t]));

```

Regla de sincronización: cada PNG nuevo requiere añadir entrada en `TILE_CATALOG` (cliente) y el `id` en `VALID_TILE_IDS` (servidor).

25. Tokens con imagen circular

Requisito

Los tokens deben mostrar una imagen de criatura recortada en círculo. Sin imagen, el token muestra el color de fondo con el nombre.

Implementación

Konva no tiene "imagen circular" nativa. La técnica es `clipFunc` en un `Group` :

```

<Group
  clipFunc={ctx => ctx.arc(CELL/2, CELL/2, radius, 0, Math.PI * 2)}
>
  <KonvaImage
    x={CELL/2 - radius}
    y={CELL/2 - radius}
    width={radius * 2}
    height={radius * 2}
  />

```

```
        image={tokenImg}  
      />  
    </Group>
```

El `clipFunc` define una trayectoria de canvas (arco completo = círculo). Todo lo que se renderiza dentro del `Group` queda recortado. No hay transformación de imagen, no hay CSS: es pura API de canvas 2D.

Selector de imagen en el menú contextual

El menú contextual del token (clic derecho) muestra una fila de miniaturas 32×32 px de `TOKEN_IMAGES`. Al hacer clic en una se asigna como `tokenImg` del token (ruta string guardada en MongoDB).

26. Generación de mapas con IA (Claude Sonnet)

Primer prototipo: demasiado simple

El primer prompt era directo: "Genera un mapa táctico de 20×15 para D&D 5e. Devuelve JSON." Claude devolvía JSON válido, pero los mapas eran monótonos: un solo tipo de suelo, sin mobiliario, tokens alineados en el centro.

Cambio de modelo: Haiku → Sonnet

El primer prototipo usaba `claude-haiku-4-5` por economía. Para una tarea creativa con razonamiento espacial, Haiku no seguía las instrucciones estructurales correctamente.

Decisión: cambiar a `claude-sonnet-4-6`. La generación de mapas es una operación manual del DM, no automática. La calidad justifica el coste.

El prompt estructurado

Se reescribió el prompt con 5 reglas explícitas:

1. ESTRUCTURA EN ZONAS – 2-4 zonas funcionales bien diferenciadas
2. MUROS Y PUERTAS – perímetro completo + muros internos + puertas en umbrales
3. VARIACIÓN DE SUELOS – mínimo 3 tipos distintos de tile
4. MOBILIARIO DENSO – mínimo 8-12 piezas de furniture
5. TOKENS TÁCTICOS – 2-6 criaturas en posiciones tácticamente interesantes

Con el mismo modelo y prompt más detallado, los mapas pasaron de rectángulos vacíos a tener zonas reconocibles, puertas, mesas agrupadas y guardias en posiciones creíbles.

Extractor de JSON robusto

Claude puede anteponer texto al JSON o añadir bloques de código Markdown. La función `extractFirstJSON` cuenta llaves para encontrar el primer objeto JSON completo:

```
function extractFirstJSON(text) {  
  const clean = text.replace(/```json\s*/gi, "").replace(/```/g, "");  
  const start = clean.indexOf("{");  
  let depth = 0, inString = false, escape = false;  
  for (let i = start; i < clean.length; i++) {  
    const ch = clean[i];  
    if (escape) { escape = false; continue; }  
    if (ch === '"' || ch === "'" || ch === "`") {  
      inString = !inString;  
      continue;  
    }  
    if (ch === '{' || ch === '}') {  
      depth += ch === '{' ? 1 : -1;  
      if (depth === 0) return clean.slice(start, i + 1);  
    }  
  }  
}
```

```

    if (ch === "\\") { escape = true; continue; }
    if (ch === '"') { inString = !inString; continue; }
    if (inString) continue;
    if (ch === "{") depth++;
    else if (ch === "}")
        if (--depth === 0) return clean.slice(start, i + 1);
    }
    return clean.slice(start);
}

```

Normalización del output

El servicio valida y normaliza antes de devolver:

1. Fuerza las dimensiones de `terrain` al tamaño solicitado.
2. Reemplaza cualquier `tileId` inválido por `"floor-stone"`.
3. Valida coordenadas de tokens para que no salgan del grid.
4. Resuelve `monsterName` → `ObjectId` en MongoDB (búsqueda case-insensitive).

El mapa generado no se guarda automáticamente

`POST /api/maps/generate` devuelve el JSON pero **no lo inserta en MongoDB**. El resultado se carga en el editor como estado local. El DM puede retocar y luego guardar (o no). Evita poblar la base de datos con mapas de prueba descartados.

27. aiMeta: guardar el prompt para regenerar

Problema

El DM genera un mapa con IA, lo ajusta a mano y luego quiere regenerar algo similar con el mismo estilo. Sin guardar el prompt, se pierde al cerrar el editor.

Solución

Se añadió el subschema `aiMeta` al modelo Map:

```

const aiMetaSchema = new mongoose.Schema({
  prompt: { type: String, default: "" },
  style: { type: String, default: "" }
}, { _id: false });

```

Cuando el DM guarda un mapa generado por IA, `aiMeta.prompt` y `aiMeta.style` se persisten. Al volver al editor, el botón "Regenerar con IA" abre el modal con el prompt y estilo pre-rellenados.

Dos modos de creación: Manual vs IA

La pantalla `MapSetup` fue rediseñada con dos opciones:

Manual — nombre, dimensiones (Small 15×10 / Medium 20×15 / Large 30×20 / XL 40×25), abre editor con lienzo en blanco.

IA — nombre, dimensiones, estilo visual (Mazmorra / Cueva / Taberna / Exterior / Templo / Barco), descripción libre. Al confirmar se llama a `POST /api/maps/generate` y el resultado se carga directamente en el editor.

Validación: intentar abrir el editor sin nombre muestra un mensaje de error en rojo bajo el campo. No se bloquea con `alert()` — el campo se marca con `border-color: red` y aparece un `<span>` de error debajo.

28. Sistema de feature flags por usuario

Motivación

El editor de mapas es una feature experimental y técnicamente compleja. No todos los usuarios deben tener acceso por defecto. Se quería un mecanismo para que el administrador conceda acceso individualmente, sin roles adicionales.

Diseño

Se añadió `features: { type: [String], default: [] }` al modelo `User`.

Por qué no un booleano `hasMapsAccess`: el array escala sin migración de esquema cuando se añadan más features experimentales.

Middleware `featureRequired`

```
export const featureRequired = (feature) => (req, res, next) => {
  if (!req.user) return res.status(401).json({ message: "No autenticado" });
  if (req.user.role === "admin" || (req.user.features ?? []).includes(feature))
    return next();
  return res.status(403).json({ message: "No tienes acceso a esta funcionalidad" });
};
```

Se aplica en todas las rutas de mapas: `router.use(authRequired, featureRequired("maps"))`.

Los admins siempre tienen acceso sin necesidad de concedérselo manualmente.

Guard en el cliente

```
const canUseMaps = (u) => u?.role === "admin" || u?.features?.includes("maps");
```

Doble protección: el servidor rechaza las peticiones y el cliente ni siquiera muestra los enlaces.

29. Panel de administración ampliado

Nuevas operaciones

Toggle isDM:

```
PUT /api/admin/users/:id/dm
body: { isDM: true | false }
```

Toggle feature: concede o revoca una feature. Usa `$addToSet` / `$pull` para evitar duplicados:

```
const update = enabled
  ? { $addToSet: { features: feature } }
```



```
    : { $pull: { features: feature } } };  
await User.findByIdAndUpdate(id, update, { new: true });
```

```
PUT /api/admin/users/:id/feature  
body: { feature: "maps", enabled: true | false }
```

UI del panel

La tabla de usuarios añadió columnas para "DM" y "Mapas". Para cada feature:

- Si el usuario es admin: muestra "siempre" (el admin siempre tiene acceso).
- Si tiene la feature: botón "× Revocar" (rojo).
- Si no la tiene: botón "✓ Conceder" (verde).

El cambio es optimista: el botón se actualiza al instante al recibir el usuario modificado del backend.

30. Lecciones adicionales (v2)

Sobre feature flags

- **Empezar con arrays, no con booleanos.** Un campo `features: [String]` escala infinitamente. La tercera feature experimental no requiere migración de esquema.
- **Los admins deben ser exentos automáticamente.** Todo middleware de feature debe tener `role === "admin" → next()` antes de comprobar el array.

Sobre Canvas y react-konva

- `listening={false}` es fundamental para capas decorativas. Sin él, la capa de grid intercepta todos los eventos de ratón antes de que lleguen al terreno — el pintado deja de funcionar.
- **Precargar imágenes antes de renderizar.** Si Konva recibe una `HTMLImageElement` que aún no ha disparado `onload`, la renderiza vacía y no se repinta al cargar. El patrón `preloadImages` garantiza un único re-render completo.
- `clipFunc` es más potente que cualquier CSS. Para recortar en círculo dentro de un canvas, la API nativa de Canvas 2D con `ctx.arc()` es la solución correcta.

Sobre generación con IA

- **Un prompt estructurado con reglas explícitas dobla la calidad del output.** El cambio de un prompt de 3 líneas a uno de 50 líneas con reglas numeradas produjo un salto de calidad visible.
- **Validar y normalizar el output de la IA en el servidor, nunca confiar en él.** Sin normalización, un `tileId` con `typo` o un token fuera del grid romperían el editor.
- **No guardar automáticamente el output generado.** El DM quiere ver el resultado antes de comprometerse. Un flujo "genera → carga en editor → guarda manualmente" es más natural.

Sobre OAuth

- **Unificar cuentas por email tiene sus riesgos.** Para sistemas públicos: requiere verificación del email local antes de fusionar.
- **Google OAuth no acepta `http://localhost` sin configuración adicional.** Hay que registrar el origen explícitamente en la consola de Google Cloud.

Sobre assets y licencias

- **CC0 es la única licencia sin fricción.** Kenney.nl tiene cientos de packs CC0 de calidad — la fuente correcta para proyectos sin gestión de atribuciones.

- **Verificar la licencia exacta por icono en game-icons.net.** La atribución colectiva "Icons by Lorc, Delapouite and contributors — game-icons.net — CC BY 3.0" cubre todos los iconos CC BY 3.0 del sitio.
- **Los datos del SRD de D&D 5e son CC BY 4.0** (propiedad de Wizards of the Coast). Permiten uso comercial con atribución.